

PROYECTO FP-DUAL
Adrián Campos Espejo
Base de Datos
Curso 2025-2026
Sistema de Gestión de Gimnasio

1. Descripción del proyecto

1.1. Introducción

Los gimnasios requieren una correcta organización para gestionar a sus socios, las actividades que ofrecen, los entrenadores y el control de las inscripciones y pagos. Una gestión ineficiente puede provocar errores en las reservas, pérdida de información o dificultades en el seguimiento de los clientes.

Actualmente, muchos gimnasios pequeños utilizan hojas de cálculo o aplicaciones genéricas que no están adaptadas a sus necesidades específicas. Esto dificulta la administración diaria y el acceso rápido a la información.

El presente proyecto tiene como objetivo el diseño e implementación de una base de datos relacional que permita gestionar de forma eficiente toda la información relacionada con el funcionamiento de un gimnasio.

1.2. Objetivos del proyecto

Los objetivos principales del proyecto son:

- Gestionar la información de los socios del gimnasio.
- Registrar entrenadores y clases disponibles.
- Controlar las inscripciones de los socios a las distintas actividades.
- Llevar un seguimiento de los pagos realizados.
- Garantizar la integridad y seguridad de los datos almacenados.

1.3. Alcance del proyecto

El sistema permitirá:

- Registrar socios, entrenadores y clases.
- Gestionar inscripciones al gimnasio y a las clases.
- Registrar pagos mensuales.
- Consultar historiales de actividades de los socios.

1.4. Beneficios del proyecto

- Centralización de la información.
- Reducción de errores administrativos.
- Acceso rápido a los datos.
- Base sólida para una futura aplicación web.

1.5. Conclusión

Este proyecto servirá como base para aplicar los conocimientos adquiridos en el módulo de Bases de Datos, utilizando un modelo relacional y su posterior implementación de la Base de Datos mediante SQL. Esta base de datos será el sistema de datos de una futura aplicación web para su gestión.

2. Cuestionario

Este cuestionario tiene como objetivo recopilar la información necesaria para definir los requisitos del sistema de gestión del gimnasio.

1. Información General

- ¿Cuántos socios tiene actualmente el gimnasio?
- ¿Existen distintas cuotas o tipos de socios (mensual, trimestral, anual)?
- ¿Cuántos entrenadores trabajan en el gimnasio?

2. Gestión de socios

- ¿Qué datos personales se registran de cada socio?
- ¿Se permite que un socio esté inscrito en varias clases?
- ¿Se controla el estado de los pagos de los socios?

3. Gestión de clases

- ¿Qué tipos de clases se ofrecen (spinning, yoga, musculación, etc.)?
- ¿Cada clase tiene un entrenador asignado?
- ¿Las clases tienen un aforo máximo?
- Calendario clases

4. Entrenadores
 - a. ¿Qué información se registra de los entrenadores?
 - b. ¿Un entrenador puede impartir varias clases?
 - c. ¿Existen horarios específicos para los entrenadores?
5. Pagos
 - a. ¿Cómo se gestionan los pagos (mensualidad, efectivo, tarjeta)?
 - b. ¿Se necesita un historial de pagos por socio?
 - c. ¿Qué ocurre si un socio no realiza el pago a tiempo?
6. Necesidades y mejoras
 - a. ¿Cuáles son los principales problemas del sistema actual?
 - b. ¿Qué funcionalidades considera imprescindibles en la base de datos?
7. Control acceso
 - a. Entradas
 - b. Salidas

3. Análisis de requisitos

3.1. Introducción

Este documento recoge los requisitos necesarios para el desarrollo de una base de datos relacional destinada a la gestión de un gimnasio, basándose en la información obtenida mediante el cuestionario realizado al responsable del centro.

3.2. Identificación de Actores

Los actores principales del sistema son:

1. Administrador del gimnasio.
2. Entrenadores.
3. Socios.

3.3. Requisitos Funcionales

Gestión de Socios

- RF-01: Registrar socios con sus datos personales.
- RF-02: Consultar y modificar información de socios.
- RF-03: Gestionar el estado de la suscripción del socio.

Gestión de Entrenadores

- RF-04: Registrar entrenadores y sus datos.
- RF-05: Asignar entrenadores a clases.
- RF-06: Consultar horarios de entrenadores.

Gestión de Clases

- RF-07: Registrar clases disponibles.
- RF-08: Controlar el aforo máximo de cada clase.
- RF-09: Gestionar inscripciones de socios a clases.

Gestión de Pagos

- RF-10: Registrar pagos realizados por los socios.
- RF-11: Consultar el historial de pagos.
- RF-12: Detectar socios con pagos pendientes.

3.4. Requisitos No Funcionales

- RNF-01: Garantizar la integridad de los datos.
- RNF-02: Control de accesos según el rol.
- RNF-03: Buen rendimiento en consultas.
- RNF-04: Copias de seguridad periódicas.

4. Requisitos de la base de datos

4.1. Definición de Tablas y Atributos

Tablas:

- Socios
- Entrenadores
- Clases
- Inscripciones
- Pagos
- Salas
- Máquinas

Tabla: socios

Del socio del gimnasio se desea saber:

- socio_id: Identificador único del socio.
- nombre: Nombre del socio.
- apellidos: Apellidos del socio.
- dni: Documento de identidad del socio.
- telefono: Número de teléfono de contacto.
- email: Correo electrónico del socio.
- tipo_suscripcion: Tipo de suscripción contratada (mensual, trimestral, anual).

Tabla: entrenadores

Del entrenador del gimnasio se desea saber:

- entrenador_id: Identificador único del entrenador.
- nombre: Nombre del entrenador.
- apellidos: Apellidos del entrenador.
- telefono: Número de teléfono de contacto.
- email: Correo electrónico del socio.
- especialidad: Área o tipo de clases que imparte el entrenador.

Tabla: clases

De la clase impartida en el gimnasio se desea saber:

- clase_id: Identificador único de la clase.

- nombre: Nombre de la clase (spinning, yoga, pilates, etc.).
- horario: Horario en el que se imparte la clase.
- aforo_maximo: Número máximo de socios permitidos en la clase.

Tabla: inscripciones

De la inscripción de un socio en una clase se desea saber:

- inscripcion_id: Identificador único de la inscripción.
- fecha_alta: Fecha en la que el socio se inscribe en la clase.
- fecha_baja:
- descripcion

Tabla: pagos

Del pago realizado por un socio se desea saber:

- pago_id: Identificador único del pago.
- fecha_pago: Fecha en la que se realiza el pago.
- importe: Cantidad económica pagada por el socio.
- metodo_pago: Forma de pago utilizada (efectivo, tarjeta, transferencia, domiciliación bancaria).
- estado: Estado del pago (pagado, pendiente).

Tabla Salas

De la sala del gimnasio se desea saber:

- sala_id: Indentificador único de la sala.
- nombre: Nombre o denominacion de la sala (por ejemplo, Sala Fitness, Sala Spinning, Sala Yoga).
- descripción: Información adicional sobre el uso o características de la sala.
- capacidad: Número máximo de personas que pueden ocupar la sala simultáneamente.

Tabla Máquinas

De la máquina disponible en el gimnasio se desea saber:

- maquina_id: Identificación único de la máquina.
- nombre: Nombre de la máquina (por ejemplo, cinta de correr, bicicleta estática, prensa).
- descripción: Información adicional sobre la máquina, como su función o características técnicas.

4.2. Definición de Relaciones

A continuación se describen las relaciones existentes entre las tablas del sistema, así como la cardinalidad de cada una de ellas.

Relación entre socios e inscripciones

- Un socio puede inscribirse en varias clases.
- Cada inscripción corresponde a un único socio.

Relación entre clases e inscripciones

- Una clase puede tener varios socios inscritos.
- Cada inscripción pertenece a una única clase.

Relación entre entrenadores y clases

- Un entrenador puede impartir varias clases.
- Cada clase es impartida por un único entrenador.

Relación entre socios y pagos

- Un socio puede realizar varios pagos a lo largo del tiempo.
- Cada pago corresponde a un único socio.

Relación entre clases y salas

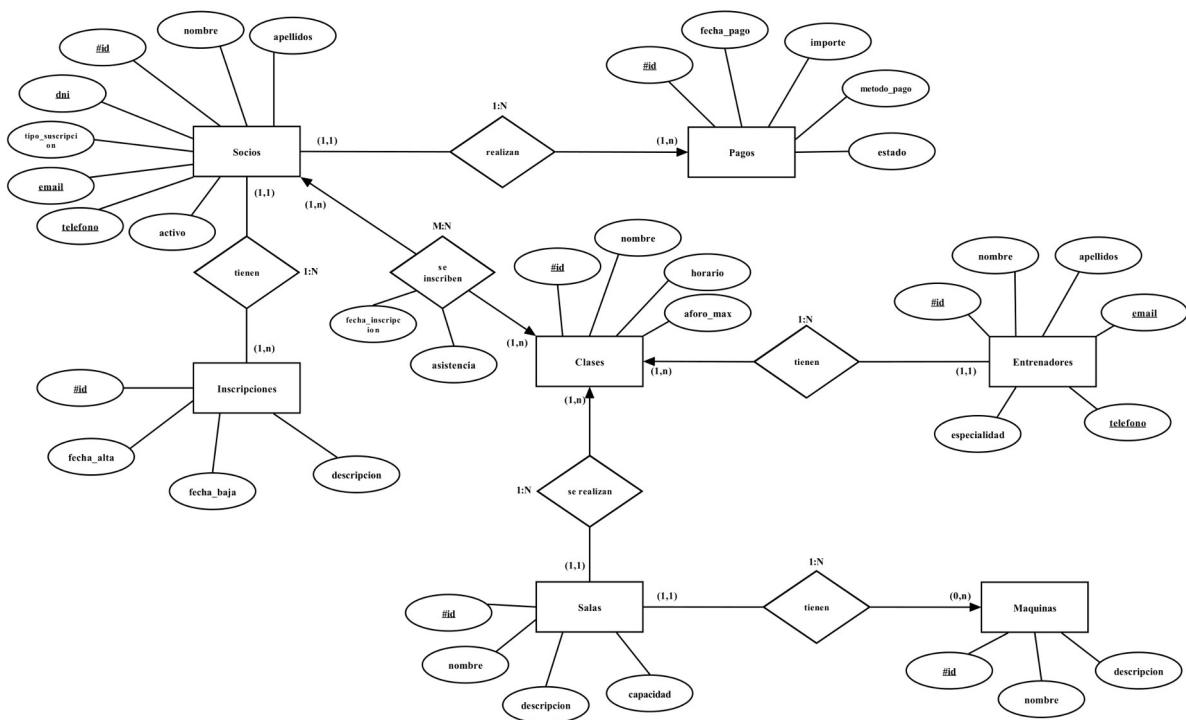
- Una clase se imparte en una sala
- En una sala se imparten varias clases

Relación entre máquinas y salas

- Una máquina solo puede estar en una sala
- En una sala puede haber varias máquinas
- Es posible que una sala no tenga máquinas

5. Diagrama Entidad-Relación

A partir de los requisitos definidos, se obtiene el siguiente Diagrama Entidad-Relación, que representa gráficamente las entidades del sistema y las relaciones existentes entre ellas.



6. Modelo relacional

Aplicando las reglas de transformación del modelo entidad-relación al modelo relacional, se obtiene el siguiente esquema:

Relación transformación de atributos y entidades

Socios (#id, nombre, apellidos, dni, email, telefono, tipo_suscripcion, activo)

Inscripciones (#id, fecha_alta, fecha_baja, descripcion)

Clases (#id, nombre, horario, aforo_max)

Pagos (#id, fecha_pago, importe, metodo_pago, estado)

Entrenadores (#id, nombre, apellidos, email, telefono, especialidad)

Salas (#id, nombre, descripcion, capacidad)

Maquinas (#id, nombre, descripcion)

Relación transformación de M:N

Socios (#id, nombre, apellidos, dni, email, telefono, tipo_suscripcion, activo)

Inscripciones (#id, fecha_alta, fecha_baja, descripcion)

Clases (#id, nombre, horario, aforo_max)

Pagos (#id, fecha_pago, importe, metodo_pago, estado)

Entrenadores (#id, nombre, apellidos, email, telefono, especialidad)

Salas (#id, nombre, descripcion, capacidad)

Maquinas (#id, nombre, descripcion)

Socios_clase (#socios_id, #clases_id, fecha_inscripcion, asistencia)

Relación transformación de 1:N

Socios (#id, nombre, apellidos, dni, email, telefono, tipo_suscripcion, activo)

Inscripciones (#id, fecha_alta, fecha_baja, descripcion)

Entrenadores (#id, nombre, apellidos, email, telefono, especialidad)

Salas (#id, nombre, descripcion, capacidad)

Socios_clase (#socios_id, #clases_id, fecha_inscripcion, asistencia)

Pagos (#id, fecha_pago, importe, metodo_pago, estado, socio_id)
socio_id -> Socios(Not Null, M:R, B:R)

Clases (#id, nombre, horario, aforo_max, entrenador_id, sala_id)

entrenador_id -> Entrenadores(Not Null, M:C, B:N)
sala_id -> Salas(Not Null, M:R, B:R)

Maquinas (#id, nombre, descripcion, sala_id)
sala_id -> Salas(Not Null, M:R, B:R)

Relación transformación de 1:1

Socios (#id, nombre, apellidos, dni, email, telefono, tipo_suscripcion, activo)

Entrenadores (#id, nombre, apellidos, email, telefono, especialidad)

Salas (#id, nombre, descripcion, capacidad)

Socios_clase (#socios_id, #clases_id, fecha_inscripcion, asistencia)

Pagos (#id, fecha_pago, importe, metodo_pago, estado, socio_id)
socio_id -> Socios(Not Null, M:R, B:R)

Clases (#id, nombre, horario, aforo_max, entrenador_id, sala_id)
entrenador_id -> Entrenadores(Not Null, M:C, B:N)
sala_id -> Salas(Not Null, M:R, B:R)

Maquinas (#id, nombre, descripcion, sala_id)
sala_id -> Salas(Not Null, M:R, B:R)

Inscripciones (#id, fecha_alta, fecha_baja, descripcion, socio_id)
socio_id -> Socios (Not Null, M:R, B:R)

Modelo Relacional Definitivo

Socios (#id, nombre, apellidos, dni, email, telefono, tipo_suscripcion, activo)

Entrenadores (#id, nombre, apellidos, email, telefono, especialidad)

Salas (#id, nombre, descripcion, capacidad)

Socios_clase (#socios_id, #clases_id, fecha_inscripcion, asistencia)

Pagos (#id, fecha_pago, importe, metodo_pago, estado, socio_id)
socio_id -> Socios (Not Null, M:R, B:R)

Clases (#id, nombre, horario, aforo_max, entrenador_id, sala_id)
entrenador_id -> Entrenadores (Not Null, M:R, B:R)
sala_id -> Salas (Not Null, M:R, B:R)

Maquinas (#id, nombre, descripcion, sala_id)
sala_id -> Salas (Not Null, M:R, B:R)

Inscripciones (#id, fecha_alta, fecha_baja, descripcion, socio_id)
socio_id -> Socios (Not Null, M:R, B:R)

7. Implementación SQL

Implementación a partir del modelo relacional anterior

```
DROP DATABASE IF EXISTS gimnasio;  
CREATE DATABASE gimnasio;  
USE gimnasio;
```

-- Tabla Socios

```
DROP TABLE IF EXISTS socios;  
CREATE TABLE IF NOT EXISTS socios (  
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(50) NOT NULL,  
    apellidos VARCHAR(100) NOT NULL,  
    dni CHAR(9) UNIQUE NOT NULL,  
    email VARCHAR(100) UNIQUE,  
    telefono VARCHAR(15),  
    tipo_suscripcion ENUM('mensual','trimestral','anual') NOT NULL,  
    activo BOOLEAN NOT NULL DEFAULT TRUE  
);
```

-- Tabla Entrenadores

```

DROP TABLE IF EXISTS entrenadores;
CREATE TABLE IF NOT EXISTS entrenadores (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    apellidos VARCHAR(100) NOT NULL,
    email VARCHAR(100),
    telefono VARCHAR(15),
    especialidad VARCHAR(100)
);

-- Tabla Salas
DROP TABLE IF EXISTS salas;
CREATE TABLE IF NOT EXISTS salas (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    descripcion TEXT,
    capacidad INT UNSIGNED NOT NULL
);

-- Tabla Clases
DROP TABLE IF EXISTS clases;
CREATE TABLE IF NOT EXISTS clases (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    horario VARCHAR(50),
    aforo_max INT UNSIGNED,
    entrenador_id INT UNSIGNED NOT NULL,
    sala_id INT UNSIGNED NOT NULL,
    FOREIGN KEY (entrenador_id) REFERENCES entrenadores(id)
        ON DELETE RESTRICT ON UPDATE RESTRICT,
    FOREIGN KEY (sala_id) REFERENCES salas(id)
        ON DELETE RESTRICT ON UPDATE RESTRICT
);

-- Tabla Maquinas
DROP TABLE IF EXISTS maquinas;
CREATE TABLE IF NOT EXISTS maquinas (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50),
    descripcion TEXT,
    sala_id INT UNSIGNED NOT NULL,

```

```

        FOREIGN KEY (sala_id) REFERENCES salas(id)
        ON DELETE RESTRICT ON UPDATE RESTRICT
    );

-- Tabla Pagos
DROP TABLE IF EXISTS pagos;
CREATE TABLE IF NOT EXISTS pagos (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    fecha_pago DATE NOT NULL,
    importe DECIMAL(8,2) NOT NULL,
    metodo_pago ENUM('efectivo','tarjeta','transferencia') NOT NULL,
    estado ENUM('pendiente','pagado','cancelado') NOT NULL DEFAULT
'pendiente',
    socio_id INT UNSIGNED NOT NULL,
    FOREIGN KEY (socio_id) REFERENCES socios(id)
    ON DELETE RESTRICT ON UPDATE RESTRICT
);

-- Tabla Inscripciones
DROP TABLE IF EXISTS inscripciones;
CREATE TABLE IF NOT EXISTS inscripciones (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    fecha_alta DATE NOT NULL,
    fecha_baja DATE,
    descripcion TEXT,
    socio_id INT UNSIGNED NOT NULL,
    FOREIGN KEY (socio_id) REFERENCES socios(id)
    ON DELETE RESTRICT ON UPDATE RESTRICT
);

-- Tabla Socios_Clase
DROP TABLE IF EXISTS socios_clases;
CREATE TABLE IF NOT EXISTS socios_clases (
    socios_id INT UNSIGNED NOT NULL,
    clases_id INT UNSIGNED NOT NULL,
    fecha_inscripcion DATE NOT NULL,
    asistencia ENUM('asistio','no_asistio') DEFAULT NULL,
    fecha_finalizacion DATE,
    activo BOOLEAN NOT NULL,
    PRIMARY KEY (socios_id, clases_id),
    FOREIGN KEY (socios_id) REFERENCES socios(id)

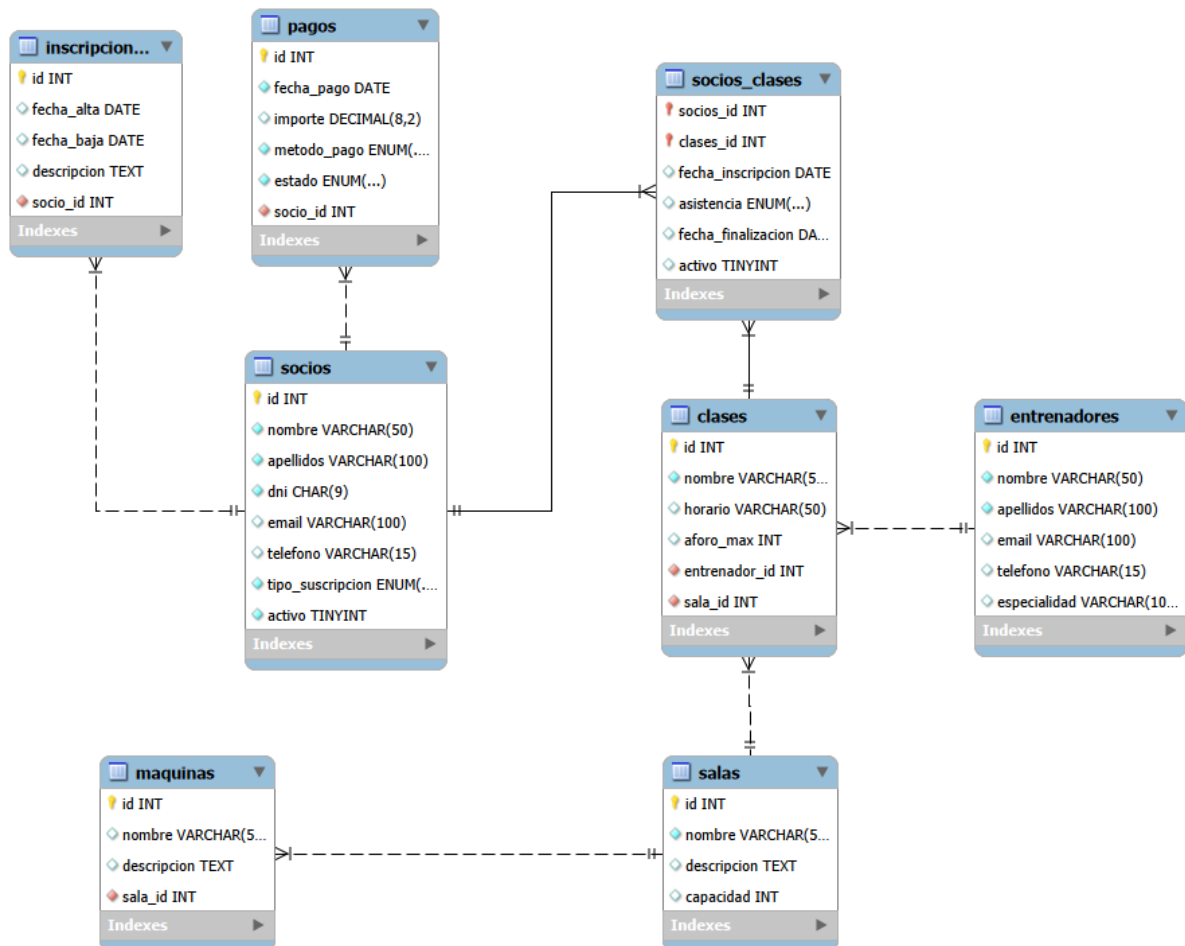
```

```

ON DELETE RESTRICT ON UPDATE RESTRICT,
FOREIGN KEY (clases_id) REFERENCES clases(id)
ON DELETE RESTRICT ON UPDATE RESTRICT
);

```

A continuación se muestra una representación gráfica generada a través de MySQL Workbench:



8. Evaluación Técnica del Proyecto

Una vez finalizado el diseño conceptual, lógico e implementación física de la base de datos, se realiza la siguiente evaluación técnica del sistema desarrollado.

8.1. Validación del Modelo Relacional

El modelo relacional obtenido a partir del diagrama entidad-relacion ha sido correctamente transformando las reglas de paso de:

- Entidades a tablas.
- Relaciones 1:N mediante claves foráneas.
- Relaciones N:M mediante tabla intermedia (socios_clases).
- Definición explícita de claves primarias en todas las tablas.

La estructura final refleja correctamente las cardinalidades definidas en el análisis de requisitos.

8.2. Integridad de Entidad

Todas las tablas disponen de:

- Clave primaria definida.
- Restricción NOT NULL en atributos obligatorios.
- Identificadores únicos mediante AUTO_INCREMENT.

Esto garantiza que no puedan existir registros sin identificar.

8.3. Integridad Referencial

Las relaciones entre tablas están implementadas mediante claves foráneas correctamente tipadas (INT UNSIGNED).

Se han definido restricciones:

- ON DELETE RESTRICT
- ON UPDATE RESTRICT

Lo que impide eliminar registros que estén siendo referenciados por otras tablas, evitando inconsistencias y garantizando coherencia en los datos.

8.4. Normalización

El diseño cumple hasta la Tercera Forma Normal (3FN):

- 1FN: No existen atributos multievaluados ni grupos repetitivos.
- 2FN: Todos los atributos dependen completamente de su clave primaria.
- 3FN: No existen dependencias transitivas entre atributos no clave.

Esto elimina redundancias y reduce el riesgo de anomalías en inserciones, actualizaciones o borrados.

8.5. Integridad de Dominio

Se ha reforzado la integridad de dominio mediante el uso de:

- ENUM para tipo_suscripcion, metodo_pago y asistencia.
- Restricciones NOT NULL en atributos obligatorios.
- Restricciones UNIQUE en dni y email.

Esto limita la introduccion de valores inválidos y mejora la calidad de los datos.

8.6. Coherencia con los Requisitos

La base de datos implementada cubre todos los requisitos funcionales definidos en el apartado 3:

- Gestión completa de socios
- Control de entrenadores y clases
- Inscripciones a actividades
- Registro e historial de pagos
- Gestión de salas y máquinas

Por tanto, el sistema diseñado responde adecuadamente a las necesidades planteadas en el análisis inicial.

9. Mina de datos

```
USE gimnasio;

-- Salas
INSERT INTO salas VALUES
(NULL, 'Sala Fitness', 'Zona de entrenamiento con pesas y máquinas', 40),
(NULL, 'Sala Spinning', 'Sala equipada con bicicletas estáticas', 25),
(NULL, 'Sala Yoga', 'Espacio para clases de yoga y pilates', 20);

-- Entrenadores
INSERT INTO entrenadores VALUES
(NULL, 'Carlos', 'Martínez López', 'carlos_gym@gmail.com', '600123456',
'Musculación'),
(NULL, 'Laura', 'Gómez Ruiz', 'laura_gym@gmail.com', '600234567', 'Yoga'),
(NULL, 'David', 'Sánchez Pérez', 'david_gym@gmail.com', '615239468',
'Spinning');

-- Socios
INSERT INTO socios VALUES
(NULL, 'Ana', 'López García', '63957482P', 'ana@gmail.com', '612458965',
'mensual', TRUE),
(NULL, 'Javier', 'Ruiz Torres', '96452136L', 'javier@gmail.com',
'659326112', 'trimestral', TRUE),
(NULL, 'Marta', 'Sanchez Díaz', '63259996D', 'marta@gmail.com',
'648552369', 'anual', TRUE),
(NULL, 'Luis', 'Fernández Romero', '23584695M', 'luis@gmail.com',
'645775321', 'mensual', TRUE),
(NULL, 'Elena', 'Morales Pérez', '45698551H', 'elena@gmail.com',
'633524995', 'mensual', TRUE);

-- Clases
INSERT INTO clases VALUES
(NULL, 'Spinning Avanzado', 'Lunes 18:00', 25, 3, 2),
(NULL, 'Yoga Relajante', 'Martes 17:00', 20, 2, 3),
(NULL, 'Entrenamiento Funcional', 'Miercoles 19:00', 30, 1, 1),
(NULL, 'Spinning Básico', '', 20, 3, 2);

-- Máquinas
```

```
INSERT INTO maquinas VALUES
(NULL, 'Cinta de correr', 'Entrenamiento cardiovascular', 1),
(NULL, 'Prensa de piernas', 'Trabajo de tren inferior', 1),
(NULL, 'Bicicleta estática', 'Cardio de bajo impacto', 1),
(NULL, 'Esterillas', 'Material para yoga y estiramientos', 3);
```

-- Inscripciones

```
INSERT INTO inscripciones VALUES
(NULL, '2025-09-01', NULL, 'Alta mensual', 1),
(NULL, '2025-09-01', NULL, 'Alta trimestral', 2),
(NULL, '2025-09-01', NULL, 'Alta anual', 3),
(NULL, '2025-09-01', '2025-12-01', 'Baja voluntaria', 4),
(NULL, '2025-09-10', NULL, 'Nueva inscripción mensual', 5);
```

-- Pagos

```
INSERT INTO pagos VALUES
(NULL, '2025-09-01', 35.00, 'tarjeta', 'pagado', 1),
(NULL, '2025-09-01', 35.00, 'transferencia', 'pagado', 2),
(NULL, '2025-09-01', 35.00, 'tarjeta', 'pagado', 3),
(NULL, '2025-10-01', 35.00, 'efectivo', 'pendiente', 1),
(NULL, '2025-09-01', 35.00, 'tarjeta', 'pagado', 5);
```

-- Socios_clases

```
INSERT INTO socios_clases VALUES
(1, 1, '2025-09-05', 'asistio', NULL, TRUE),
(2, 2, '2025-09-06', 'asistio', NULL, TRUE),
(3, 1, '2025-09-07', 'no_asistio', NULL, TRUE),
(1, 3, '2025-09-08', 'asistio', NULL, TRUE),
(5, 2, '2025-09-12', NULL, NULL, TRUE);
```